Software Requirements Specification Document - UVSim

By: Ryan Black

1. Introduction
   1.1 Purpose:
   Define specific needs of the UVSim system and how to determine if the product has been completed and successfully implemented.
   1.2 Intended Audience:
   Client of UVSim, Developers of UVSim, Student using UVSim.
   1.3 Intended Use:
   Use for reviewing of requirements both functional and non-functional requirements and be a resource of definitions.
   1.4 Product Scope:
   Scope would be specific for the application of UVSim including main executable loop, memory class, UVSim functions, and GUI.
   1.5 Definitions and Acronyms:

   UVSim: Program for executing user provided files

   Accumulator: Place in which values may be stored and used for UVSim Functions

   Functions: Controls and operations that a user provided file can use to run a system.

2. Descriptions
   2.1 User Needs:
   Needs to allow users to understand the system stores and execute values.
   Needs to allow both Teacher and student to test files.
   2.2 Assumptions and Dependencies:
   Needs to have prior knowledge on how files need to be written and what opcode does what.
3. System Features and Requirements
   3.1 Functional Requirements:
   3.1.1 The system should take a file provided from a user for data.
   3.1.2 The system shall take the file and parse the information into class memory.

3.1.3 The system will loop till a halt command is given in user provided file.
3.1.4 The system should have a case switch that will pick different functions based on the user's given operation.
3.1.5 Users must need feedback if the system hard crashes.
3.1.6 The accumulator will truncate numbers to prevent overflow.
3.1.7 Inputs will be tested for valid ranges when user gives an input.
3.1.8 If input is invalid will loop till valid input is received.
3.1.9 Must have a UI that is well-labeled.
3.1.10 System will check to make sure that inputs are provided to functions with correct data.
3.1.11 The Control operators will function separate of the class functions.
3.1.12 The simulation will have a Program Counter to make sure that the system knows what step to execute
3.1.13 System will have unit tests that fail and succeed in knowing different error messages.
3.1.14 System will ensure that all inputs and parsing are valid.
3.1.15 System will have 100 memory locations that can be accessed in a memory class.

3.2 Non-Functional Requirements:

3.2.1 System will be able to run and execute provided file in 2 seconds with valid inputs.
3.2.2 The system will be able to be distributed and work on multiple operating systems.
3.2.3 The system will prevent users from accessing private system information.